

嵌入式计算机系统与实验

Shuiyuan@Noroshi

1 ADC

绝大部分的现实生活中的信号都是在时域上连续的, 如电压等, 但大部分计算机所读取的数字信号则是离散的. 为此我们需要一个 ADC(即模拟信号至数字信号转换器), 来完成这个工作. 其输出精度是 12 位, 也就是输出最大值为 4095. 其需要两个参考电压, 也就是最小值和最大值, 之后输入的信号强度位于二者之间的就会被转换为 0~4095 中的对应值.

ADC 的相关寄存器主要包括 ADC12CTL0(主要控制寄存器, 用来启动停止, 配置参数等), ADC12CTL1(选择转换模式, 通道, 时钟源等), ADC12IFG(中断标志寄存器, 标志是否完成转换), ADC12IE(控制中断是否启用), ADC12IV(中断向量寄存器)等.

1.1 ADC 实验

```
/*
 * main.c
 */

#include<msp430f6638.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include "dr_lcdseg.h"
void main(void)
{
    initLcdSeg();
    WDTCTL = WDTPW + WDTHOLD;
    P4DIR |= BIT5 + BIT6 + BIT7;
    P5DIR |= BIT7;
    P8DIR |= BIT0;
    ADC12CTL0 |= ADC12MSC;
    ADC12CTL0 |= ADC12ON;
    ADC12CTL1 |= ADC12CONSEQ1 ;
    ADC12CTL1 |= ADC12SHP;
    ADC12MCTL0 |= ADC12INCH_15;
    ADC12CTL0 |= ADC12ENC;
    volatile unsigned int value = 0;
    while(1)
    {
        ADC12CTL0 |= ADC12SC;
        value = ADC12MEM0;
        LCDSEG_DisplayNumber(value, 0);
        if(value > 5)
            P4OUT |= BIT5;
        else
            P4OUT &= ~BIT5;
        if(value >= 800)
            P4OUT |= BIT6;
    }
}
```

```

else
    P4OUT &= ~BIT6;
if(value >= 1600)
    P4OUT |= BIT7;
else
    P4OUT &= ~BIT7;
if(value >= 2400)
    P5OUT |= BIT7;
else
    P5OUT &= ~BIT7;
if(value >= 3200)
    P8OUT |= BIT0;
else
    P8OUT &= ~BIT0;
}
}

```

这部分代码实现了 ADC 读取外部的电压源的电压大小, 并在不同的电压强度下点亮不同数量的 LED 灯, 以及在 LCD 上显示转换的数值。

我们来分析一下这部分代码, 刚开始就是各类初始化, 包含初始化 LCD 模块, 关闭看门狗, 打开 LED 灯对应引脚的输出模式, `ADC12CTL0 |= ADC12MSC;` 启用了 ADC 的多次采样和转换的模式, `ADC12CTL0 |= ADC12ON;` 表示打开 ADC 的电源, `ADC12CTL1 |= ADC12CONSEQ1;` 表示设置 ADC 为单通道重复转换模式, `ADC12CTL1 |= ADC12SHP;` 选择采样的保持信号的时钟来源, `ADC12MCTL0 |= ADC12INCH_15;` 将通道 15 设置为 ADC 的输入通道, `ADC12CTL0 |= ADC12ENC;` 则正式启动了 ADC 转换。

在主循环内, `ADC12CTL0 |= ADC12SC;` 表示其正式开始了一次转换, 并将转换的值存储在了 `ADC12MEM0` 寄存器中, 第二行我们就把其值赋给了我们实际的程序内变量 `volatile unsigned int value = 0;` 的 `volatile` 的意思就是指示编译器不对此变量的相关操作进行优化, 因为是需要与硬件交互的变量。

之后的部分就是 LED 和 LCD 的输出, 不多赘述。

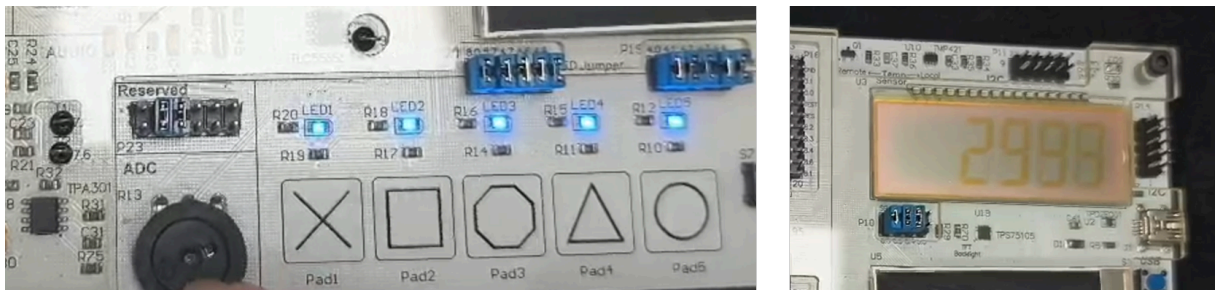


Figure 1: LED 的现象与 LCD 的现象, 具体见视频 lab8-1-1 和 lab8-1-2.

2 DAC

DAC 就是反过来, 将数字信号转换为模拟信号。

2.1 示例实验

```

#include<msp430f6638.h>

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;
    P7DIR |= BIT6;
    P7SEL |= BIT6;
    DAC12_0CTL0 |= DAC12IR;
    DAC12_0CTL0 |= DAC12SREF_1;
    DAC12_0CTL0 |= DAC12AMP_5;
}

```

```
DAC12_CTL0 |= DAC12CALON;  
DAC12_CTL0 |= DAC120PS;  
DAC12_CTL0 |= DAC12ENC;  
DAC12_DAT = 0xFFF;  
  
__bis_SR_register(LPM4_bits);  
}
```

这个就是最简单的, 向 DAC 中写入一个固定的电压值的程序, 其作用就输出电压. 前面 DAC 控制寄存器的配置也就是打开模块, 选择电压和输出方式, 使能校准, 使之可以正常输出正确的信号.

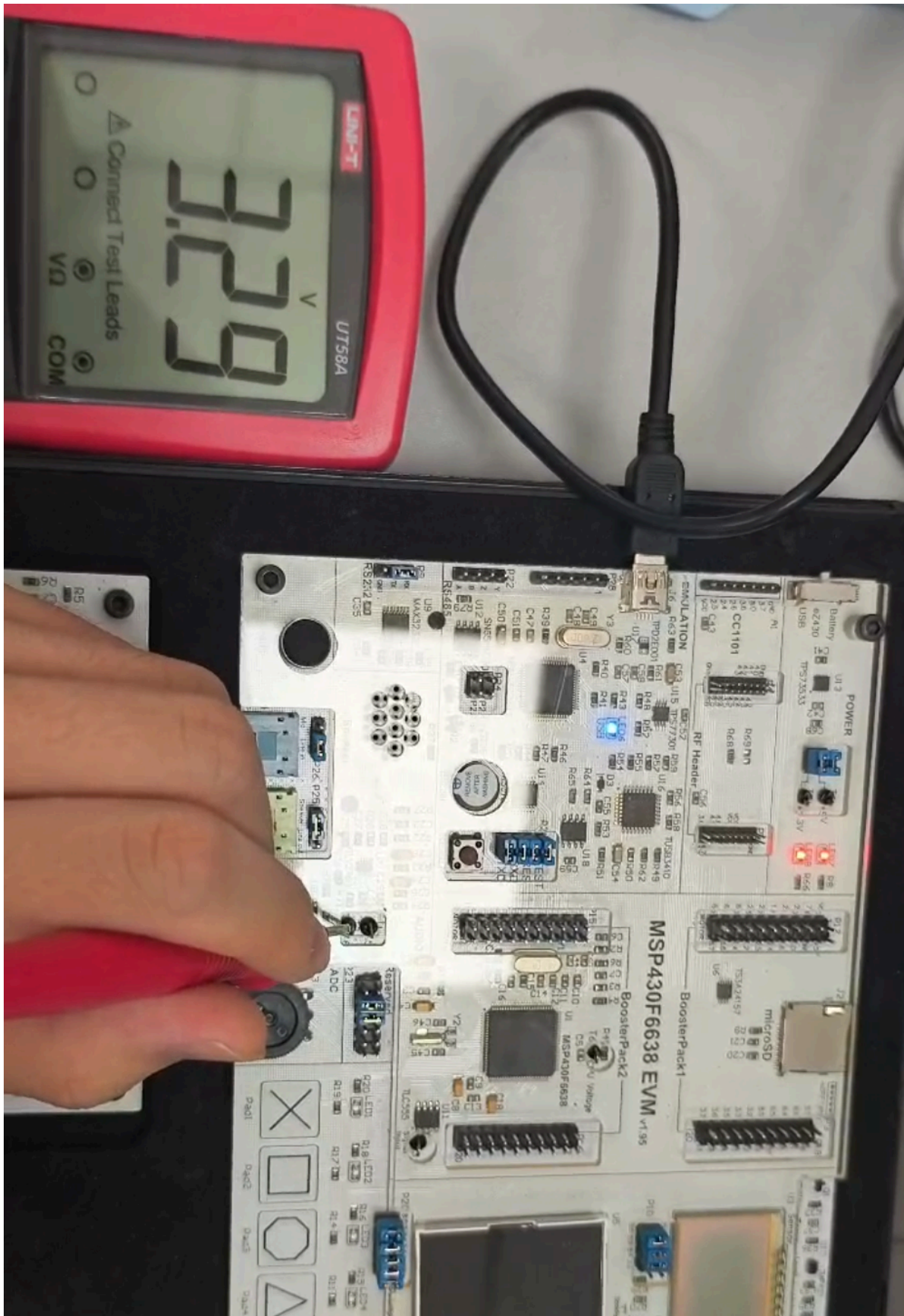


Figure 2: 输出电压, 这里因为我们采用的参考电压是 3.3v, 然后我们输出的数字信号是最大值(0xffff), 所以结果就是 3.29v

2.2 拓展实验

这部分实验我们实现了输出正弦波

```
#include <msp430f6638.h>

#define PI 3.1415926535
#define SAMPLE_POINTS 10
#define TIMER_INTERVAL 10

const unsigned int sine_table[SAMPLE_POINTS] = {
    2048,
    3251,
    3995,
    3995,
    3251,
    2048,
    845,
    101,
    101,
    845
};

volatile unsigned int sample_index = 0;

void main(void) {
    WDTCTL = WDTPW | WDTHOLD;

    P7DIR |= BIT6;
    P7SEL |= BIT6;
    DAC12_0CTL0 = DAC12IR | DAC12SREF_1 | DAC12AMP_5 | DAC120PS | DAC12ENC;
    DAC12_0CTL1 = DAC12LSEL_0;
    DAC12_0DAT = sine_table[0];

    while (1) {
        __delay_cycles(10000);
        DAC12_0DAT = sine_table[sample_index];
        sample_index = (sample_index + 1) % SAMPLE_POINTS;
    }
}
```

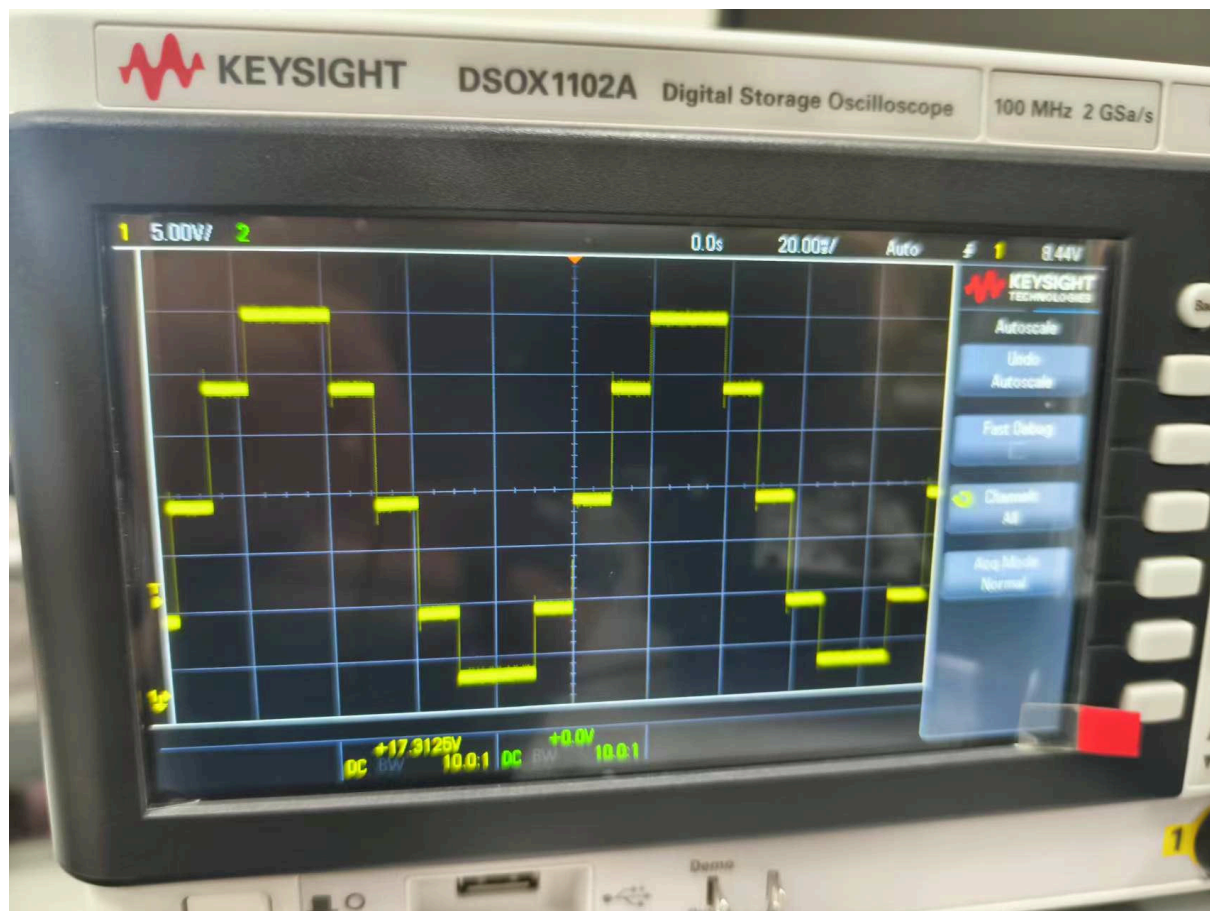


Figure 3: 输出正弦波, 如果采样点多点会更逼真一点.